

ECG Model Compression for Embedded Inference

Aleksandar Toslev Luka Zotovikj

Universität Heidelberg

Beginner's Practical: Neural Networks from Scratch September 2026

Abstract

This project asks whether a heartbeat classifier implemented from first principles can be compressed into a practical microcontroller model without losing its class-level behaviour. We implemented a five-class one-dimensional convolutional neural network (CNN) directly in NumPy, including convolution, pooling, dense layers, dropout, weighted softmax cross-entropy, backpropagation, and Adam. The supplied MIT-BIH heartbeat CSV data are highly imbalanced, so the training study combines capped class weights, mild rare-class augmentation, and a comparison of three mini-batch construction methods under otherwise fixed settings. The selected float32 model has 11,173 parameters and reaches 95.26% test accuracy with a macro-F1 of 0.8050. Full INT8 post-training quantization reduces the exported model from 90,361 to 23,576 bytes (73.91%) while retaining 95.12% accuracy and a macro-F1 of 0.8055. The quantized model executes on an ESP32 with an 80 KiB tensor arena and 52.090 ms mean inference time per stored, pre-segmented beat. A balanced 25-beat hardware run classifies 21/25 correctly; complete predictive performance is assessed separately on the 21,892-beat PC test set. The result is an end-to-end educational TinyML pipeline from explicit NumPy training to measured physical deployment, not a clinically validated medical system.

Keywords: ECG classification, 1D CNN, class imbalance, INT8 quantization, TensorFlow Lite Micro, ESP32, TinyML

1 Motivation and Related Work

Automatic analysis of electrocardiograms (ECGs) can support the detection of abnormal cardiac rhythms, but embedded monitoring platforms impose strict memory and compute limits. The MIT-BIH Arrhythmia Database remains a standard reference collection for arrhythmia research [1], distributed through the PhysioNet research resource [2]. This project uses the pre-segmented, 187-sample heartbeat representation released with the five-class deep-learning study of Kachuee et al. [3]. Each input is therefore one already isolated beat, not a continuous ECG recording.

CNNs learn local patterns through shared filters [4]; one-dimensional variants have been applied directly to ECG waveforms and real-time monitoring [5]. Our emphasis differs from work seeking the highest clinical benchmark score: the complete forward and backward passes are made inspectable in NumPy, and the trained parameters are then carried into a reproducible embedded deployment. The severe class imbalance also matters. Re-weighting, re-sampling, and data-level methods are established responses to imbalanced learning [6, 7], but their interaction must be tested rather than assumed beneficial.

The final embedded pipeline uses a naturally compact CNN followed by full INT8 post-training quantization (PTQ). Integer-only inference reduces model storage and supports efficient execution on constrained devices [8]. The main contributions are:

1. an explicit NumPy implementation of a compact 1D CNN and its training operations;
2. an imbalance study that exposes harmful overcorrection when multiple balancing mechanisms are combined;
3. a full-test-set float32-to-INT8 comparison with class-level metrics and artifact sizes; and
4. physical TensorFlow Lite Micro deployment on ESP32 with measured memory and latency.

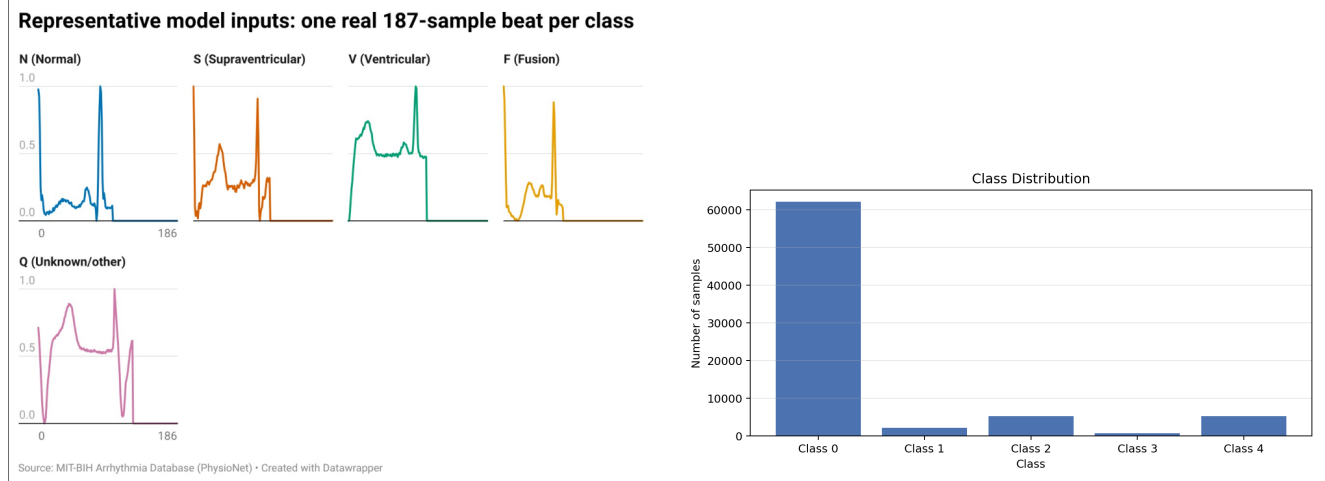
2 Dataset and Preprocessing

The submitted code package expects `data/processed/mitbih_train.csv` and `mitbih_test.csv`. Each row contains 187 normalized amplitude values followed by a label $y \in \{0, 1, 2, 3, 4\}$. The five categories follow the grouped representation used by Kachuee et al. [3]: N (normal), S (supraventricular), V (ventricular), F (fusion), and Q (unknown/other). Values in the supplied CSV files already lie in the intended range, so the selected run uses `normalize=None`; the loader nevertheless supports standard and per-sample normalization for controlled experiments.

The fixed test CSV contains 21,892 beats. A stratified 15% validation split is taken from the 87,554-beat training CSV with seed 42, leaving 74,420 beats for fitting and 13,134 for validation. The original test file is never used for optimization, augmentation, or quantization calibration.

Table 1: Training counts before augmentation. Class N contains roughly 113 times as many beats as class F.

Class	Group	Training beats	Share (%)
0	N: normal	61,600	82.77
1	S: supraventricular	1,889	2.54
2	V: ventricular	4,920	6.61
3	F: fusion	545	0.73
4	Q: unknown/other	5,466	7.34



(a) One real 187-sample model input per class. Visualization created by the authors with Datawrapper [9]. (b) Training distribution after the stratified split and before augmentation.

Figure 1: The classifier receives short pre-segmented beats, while the learning problem is dominated numerically by normal beats. Source data: MIT-BIH/PhysioNet [1, 2].

The loader validates finite values, feature count, label range, and class coverage. The input tensor is reshaped from (187) to (187, 1) because every convolution consumes (batch, time, channels). All final metrics use the untouched fixed test CSV. Because the provided split is beat-based rather than demonstrated patient-independent, it must not be interpreted as clinical generalization to unseen patients.

3 From-Scratch 1D CNN

3.1 Architecture

The selected network is intentionally small. Three convolution blocks learn progressively richer local morphology; global average pooling replaces a large flatten-and-dense head. This choice changes a $46 \times 64 = 2,944$ feature vector into only 64 averaged features and avoids roughly 92,000 additional dense-layer weights, following the compactness motivation of global average pooling [10]. Dropout with rate 0.2 is active only during training [11].

Table 2: Final architecture. Parameter counts include biases; ReLU, pooling, and dropout have no trainable parameters.

Stage	Operation	Output shape	Parameters
Input	one pre-segmented heartbeat	187×1	0
1	Conv1D 16, kernel 7, same; ReLU; MaxPool 2	93×16	128
2	Conv1D 32, kernel 5, same; ReLU; MaxPool 2	46×32	2,592
3	Conv1D 64, kernel 3, same; ReLU	46×64	6,208
4	Global average pooling	64	0
5	Dense 32; ReLU; dropout 0.2	32	2,080
6	Dense class scores; softmax	5	165
Total			11,173

The resulting forward pass requires approximately 543,856 multiply-accumulate operations per beat. This is small enough to investigate manually and later execute on the microcontroller.

3.2 Manual forward and backward passes

For input x , kernel W , bias b , and same-padding offset p , the implemented convolution is

$$z_{n,t,o} = b_o + \sum_{k=0}^{K-1} \sum_{c=0}^{C_{in}-1} x_{n,t+k-p,c} W_{k,c,o}, \quad (1)$$

with out-of-range positions treated as zero. ReLU stores its activation mask; max pooling stores the winning index; dense and convolution layers retain the tensors needed for local gradients. Backpropagation then propagates $\partial\mathcal{L}/\partial z$ in reverse order and accumulates explicit gradients for every parameter.

The logits z_i are converted to probabilities by a numerically stable softmax,

$$p_i = \frac{\exp(z_i - \max_j z_j)}{\sum_k \exp(z_k - \max_j z_j)}. \quad (2)$$

The implementation includes weighted cross-entropy, dropout, SGD, and Adam. The selected run uses Adam with learning rate 10^{-3} [12]. TensorFlow/Keras does not train this model: after NumPy training, an equivalent Keras graph is constructed and receives the learned NumPy weights solely as an export bridge to TensorFlow Lite.

4 Training and Class Imbalance

All sampling experiments use 20 epochs, batch size 128, seed 42, Adam, learning rate 10^{-3} , and identical architecture and augmentation settings. Accuracy alone would reward the dominant N class, so selection also considers macro-F1, per-class precision/recall/F1, and confusion matrices.

4.1 Capped class weighting and augmentation

Inverse-frequency weights are computed as

$$w_c = \min\left(4, \frac{N}{CN_c}\right), \quad (3)$$

where N_c is the number of training beats in class c and $C = 5$. The cap prevents the rarest classes from producing unboundedly stronger gradients. The final weights are (0.242, 4.000, 3.025, 4.000, 2.723) for classes 0–4.

Only classes below the rare threshold are augmented toward 2,000 training examples. S receives 111 generated beats and F receives 1,455. Each copy uses mild amplitude scaling in $[0.95, 1.05]$, small Gaussian noise, a baseline shift in $[-0.01, 0.01]$, and a temporal shift of at most two samples. These changes are intended to preserve heartbeat morphology; augmentation is applied only to the training partition.

4.2 Mini-batch construction comparison

We compared natural shuffled mini-batches with inverse-frequency weighted sampling and approximately balanced mini-batches. The latter two methods failed under the already weighted and augmented setup (Table 3). This does not show that re-sampling is generally ineffective; it shows that, in this configuration, its interaction with capped loss weighting and augmentation overcorrected the data distribution and biased predictions away from the majority class.

Table 3: Comparison of three mini-batch construction methods. Architecture, seed, optimizer, loss weighting, and augmentation are held fixed; natural shuffle is selected.

Sampling	Test loss	Accuracy (%)	Macro-F1	Weighted-F1
Natural shuffle	0.1950	95.26	0.8050	0.9536
Weighted	1.9191	16.95	0.3022	0.0973
Balanced batches	1.8431	17.49	0.3155	0.1165

5 Full INT8 Post-Training Quantization

PTQ is applied after the selected NumPy model has finished training. The Keras export is passed to the TensorFlow Lite converter, which observes 200 representative training beats to estimate activation

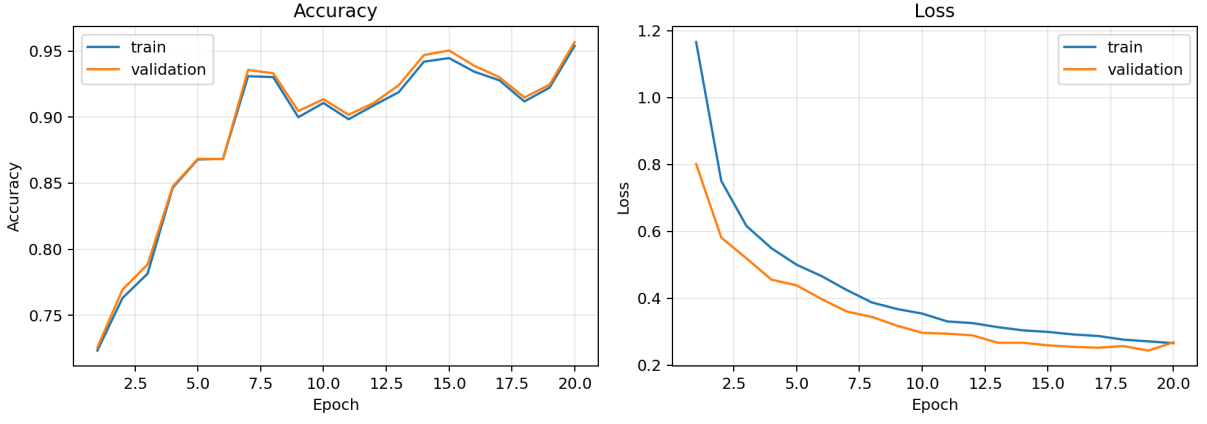


Figure 2: Accuracy and weighted loss for the selected natural-shuffle run. Training and validation trajectories remain close over 20 epochs; the non-monotonic accuracy reflects weighted optimization and mild on-the-fly augmentation.

ranges. TensorFlow’s full-integer workflow requires such calibration data for intermediate tensors [13]. Conversion is restricted to INT8 built-in operations, and both model input and output use signed 8-bit integers.

For real value r , integer value q , scale S , and zero-point Z , affine quantization approximates

$$r \approx S(q - Z), \quad q = \text{clip}(\text{round}(r/S) + Z, -128, 127). \quad (4)$$

Rounding and clipping can alter decisions close to class boundaries, but integer arithmetic reduces storage and supports microcontroller inference [8].

Table 4: Authoritative full-test-set comparison on 21,892 beats. The tiny macro-F1 increase after quantization is numerical variation, not evidence that PTQ improves the classifier.

Metric	Float32 export	Full INT8
Test accuracy	0.9526	0.9512
Test loss	0.1950	0.2145
Macro precision	0.7775	0.7779
Macro recall	0.8555	0.8514
Macro-F1	0.8050	0.8055
Weighted-F1	0.9536	0.9521
Trainable parameters	11,173	11,173
Artifact size (bytes)	90,361	23,576

Accuracy falls by only 0.14 percentage points while exported model size falls by 73.91%. The parameter count is unchanged: PTQ changes numerical representation rather than topology.

6 ESP32 Deployment

TensorFlow Lite Micro is designed to execute neural-network graphs under embedded-system constraints [14]. The final pipeline is

$$\text{NumPy weights} \rightarrow \text{Keras export} \rightarrow \text{INT8 TFLite} \rightarrow \text{C array} \rightarrow \text{ESP-IDF/TFLM}.$$

The firmware in `esp32/esp_idf_ecg_classifier/` allocates an 80 KiB tensor arena, loads the 23,576-byte model, quantizes 187 stored amplitudes with the model input scale and zero-point, invokes the interpreter, and dequantizes five output scores for display. The tested ESP32 uses a 160 MHz CPU configuration and 4 MB flash; these are consistent with the platform capabilities documented by Espressif [15].

Mean inference latency is 52.090 ms per beat (minimum 52.015 ms, maximum 52.659 ms). All four errors in this small subset are minority beats classified as N. This run proves that the exported graph executes on physical hardware and provides a latency measurement; it is not an accuracy estimate comparable to the complete test set.

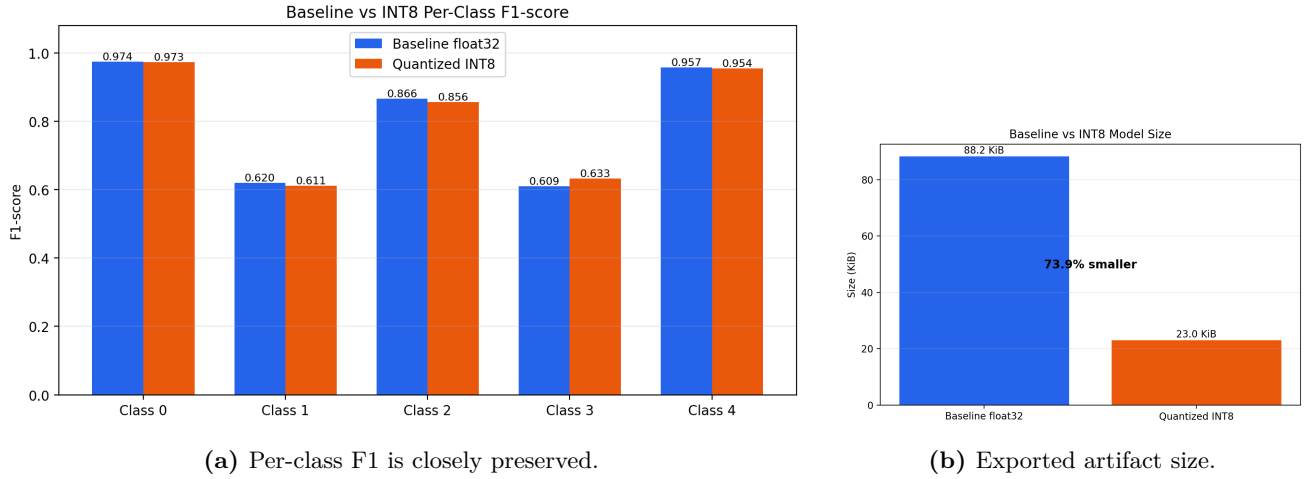


Figure 3: Float32 versus full INT8. Classes 0–4 correspond to N, S, V, F, and Q.

Table 5: Balanced physical-hardware check: five reproducibly sampled beats per class, without replacement, seed 42.

	N	S	V	F	Q	Overall
Correct / tested	5/5	4/5	5/5	3/5	4/5	21/25
Subset recall (%)	100	80	100	60	80	84

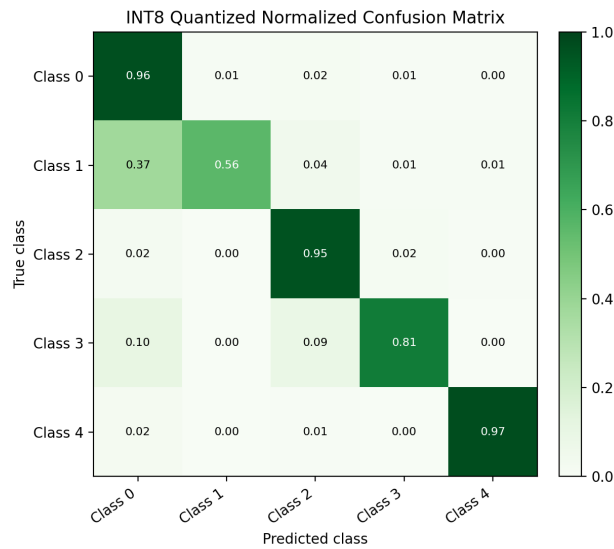


Figure 4: Row-normalized confusion matrix for the complete PC-side INT8 evaluation. Classes 0–4 are N, S, V, F, and Q. The principal remaining weakness is S: approximately 37% of true S beats are predicted as N.

7 Discussion, Project Progression, and Limitations

The central result is not simply a high total accuracy. It is the preservation of class-level behaviour after a complete representation and deployment change: an explicitly trained NumPy model becomes a fully integer TensorFlow Lite graph and runs on a physical ESP32. Global average pooling is important to this outcome because the architecture begins compact; INT8 PTQ then reduces storage without relying on sparse formats or custom kernels.

The experimental history also contains an important negative result: the three-way mini-batch comparison shows that stacking imbalance remedies can be actively harmful. Weighted loss, augmentation, and re-sampling must be evaluated as an interacting system rather than assumed to provide additive benefits. The validated end-to-end result therefore retains natural shuffle together with capped class weighting, mild augmentation, and PTQ.

Table 6: How the experimental pipeline evolved into the final validated system. Historical artifacts document intermediate and unsuccessful training configurations.

Phase	Role and outcome
Baseline pipeline	Established data loading, manual training scaffolding, metrics, artifact saving, and compression plumbing; exposed the need for class-aware evaluation.
CNN bring-up	Implemented Conv1D, pooling, dense layers, explicit gradients, evaluation, and Keras weight export; exposed weak minority-class recall.
Imbalance study	Added capped class weights, morphology-preserving augmentation, and three sampling modes; demonstrated severe overcorrection for weighted/balanced sampling.
PTQ and ESP32	Selected natural shuffle, converted the final 11,173-parameter CNN to full INT8, and validated the same artifact on the full PC test set and physical ESP32.

The remaining class behaviour is uneven. N, V, and Q have strong F1, whereas S and F remain more difficult. Macro-F1 therefore communicates more than accuracy. The small INT8 change in macro-F1 should not be interpreted as improvement; individual rounding-induced prediction changes can help some examples and hurt others.

There are important scope limitations. Inputs are stored, pre-segmented heartbeats rather than live continuous ECG. The firmware does not acquire an analogue signal, filter noise, detect R-peaks, segment beats, reject unfamiliar inputs, or manage confidence. The provided CSV split is not demonstrated patient-independent, the selected architecture was not exhaustively searched, and the 25-beat hardware run is deliberately small. Consequently, this work is an academic TinyML prototype, not a medical device or a clinically validated diagnostic system. Future work should add patient-wise evaluation, repeated-run uncertainty, live acquisition and segmentation, confidence-based rejection, and broader architecture/latency comparisons.

8 Code Documentation and Reproducibility

The report follows the submitted code package in execution order (Table 7). The selected float32 model and metrics are stored in `results/sampling_sweep_cap_4/shuffle/`; the final INT8 model and full-test evaluation are in `results/final_candidate_cap_4/`; comparison figures are in `results/final_cap4_comparison/`; and hardware claims are supported by `results/esp32_final_cap_4/`. The historical folder name `final_candidate_cap_4/` is retained for traceability, but it contains the final submitted INT8 artifact.

Table 7: Direct mapping between report stages and submitted code.

Report stage	Submitted implementation / evidence
Data and validation	src/data/mitbih_csv.py, src/data/visualization.py
Manual CNN and gradients	src/models/cnn1d.py
Training, weighting, augmentation, sampling	src/train_cnn.py
Evaluation and plots	src/evaluation/
Full INT8 conversion	src/compression/quantize_tflite.py
TinyML model/sample export	src/export_tinyml.py, src/export_ecg_samples.py
Physical firmware	esp32/esp_idf_ecg_classifier/
Results guide	results/README.md
Final float32 evidence	results/sampling_sweep_cap_4/shuffle/
Final INT8 evidence	results/final_candidate_cap_4/
Comparison and hardware	results/final_cap4_comparison/, results/esp32_final_cap_4/

After installing the dependencies from requirements.txt and placing the two CSV files under data/processed/, the selected training and conversion can be reproduced from the root of the extracted code ZIP:

Listing 1: Core commands for the selected cap-4 model and full INT8 export.

```
python -m src train --data-dir data/processed \
--output-dir results/reproduced_cap4 \
--epochs 20 --batch-size 128 --optimizer adam \
--learning-rate 0.001 --normalize none --seed 42 \
--class-weights balanced --class-weight-cap 4 \
--sampling-strategy shuffle --augment-rare-classes \
--rare-target-count 2000

python -m src quantize \
--model results/reproduced_cap4/tiny_ecg_cnn.keras \
--data-dir data/processed \
--output results/reproduced_cap4/tiny_ecg_cnn_int8.tflite \
--normalize none --representative-samples 200

python -m src final-compare \
--baseline-dir results/sampling_sweep_cap_4/shuffle \
--int8-dir results/final_candidate_cap_4 \
--output-dir results/final_cap4_comparison \
--esp32-latency-ms 52.09 --tensor-arena-kib 80
```

The intermediate results/improved_cnn/ directory records an exploratory Keras-based CNN that reached 90.91% accuracy and 0.7327 macro-F1. It is retained to show the development progression, but it is not the final from-scratch model and must not be used for the report’s final numerical claims. The submitted ZIP should preserve the ZIP-relative paths, requirements.txt, results/README.md, the final result summaries, and the ESP-IDF README. The submission ZIP remains the authoritative frozen copy.

9 Work Distribution and Use of LLMs

9.1 Work distribution

The project was developed collaboratively, with both authors contributing an equal overall share (50% each). To make the individual contributions identifiable, the descriptions below indicate the areas in which each author took a stronger initial or coordinating role. They should not be interpreted as exclusive ownership: implementations and experiments were regularly reviewed, tested, debugged, and extended by the other author. This iterative workflow meant that major components typically evolved through contributions from both authors rather than being implemented independently by one person.

Luka Zotovikj (50%). His primary contributions included the initial development of the from-scratch CNN and training infrastructure, including the core forward and backward operations and integration of the training pipeline. He took a leading role in the class-imbalance investigation, including weighting, augmentation, alternative mini-batch sampling strategies, and analysis of their effects on minority-class performance. These components were subsequently tested, debugged, modified, and extended jointly. He also contributed to evaluation, result interpretation, visualizations, documentation, poster preparation,

and the final report.

Aleksandar Toslev (50%). His primary contributions included the iterative development and refinement of the model and experimental pipeline, including validation of the CNN implementation, training behaviour, evaluation procedures, and comparative experiments. He took a leading role in the later compression and embedded-deployment stages, including full INT8 post-training quantization, the float32–INT8 comparison, TinyML export, and ESP32/TensorFlow Lite Micro integration and hardware evaluation. These components were likewise reviewed, tested, debugged, and extended jointly. He also contributed to the class-imbalance experiments, result interpretation, visualizations, documentation, poster preparation, and the final report.

Overall contribution: Aleksandar Toslev 50% — Luka Zotovikj 50%.

9.2 Use of large language models

Large language models were used as supporting and learning tools throughout the project. Their role included discussing possible implementation approaches, explaining unfamiliar concepts and APIs, suggesting debugging strategies, reviewing code for potential issues or improvements, and assisting with code readability, documentation structure, literature discovery, wording, and LaTeX formatting. They were also used as an interactive resource for considering alternative approaches during development rather than as a substitute for implementing or evaluating the project.

All implementation decisions were reviewed and understood by the authors, and generated suggestions were adapted, tested, debugged, or rejected based on their behaviour within the actual project. The authors independently executed the training and evaluation experiments, performed the model conversion and hardware deployment, inspected the resulting outputs, consulted the relevant primary sources, and verified the numerical results against the submitted result files and ESP32 measurements. No experimental result or measurement was generated or substituted by an LLM. Responsibility for the implementation, experimental methodology, interpretation, citations, and final conclusions remains with the authors.

10 Conclusion

We implemented and trained a complete 1D CNN directly in NumPy for five-class classification of pre-segmented MIT-BIH heartbeats. The project made convolution, pooling, dense layers, dropout, weighted softmax cross-entropy, backpropagation, and Adam explicit, then addressed severe imbalance through capped loss weights, mild rare-class augmentation, and a comparison of three mini-batch construction methods under otherwise fixed settings. Natural shuffle was the only successful method under the combined setup, reaching 95.26% float32 test accuracy and 0.8050 macro-F1.

Full INT8 PTQ reduced the exported artifact from 90,361 to 23,576 bytes, a 73.91% reduction, while retaining 95.12% accuracy and 0.8055 macro-F1. The same model executed on an ESP32 using an 80 KiB tensor arena with 52.090 ms mean latency per beat. These results establish a reproducible path from first-principles neural-network training to physical embedded inference. The result is best described as the strongest tested deployment trade-off within this project, not a globally optimal classifier or a medical system. Its most meaningful next step is evaluation on patient-independent data followed by live acquisition, filtering, segmentation, and uncertainty handling.

References

- [1] G. B. Moody and R. G. Mark, “The impact of the MIT-BIH arrhythmia database,” *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.
- [2] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, “PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals,” *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.
- [3] M. Kachuee, S. Fazeli, and M. Sarrafzadeh, “ECG heartbeat classification: A deep transferable representation,” in *2018 IEEE International Conference on Healthcare Informatics (ICHI)*, 2018, pp. 443–444.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] S. Kiranyaz, T. Ince, and M. Gabbouj, “Real-time patient-specific ECG classification by 1-d convolutional neural networks,” *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016.
- [6] H. He and E. A. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

- [7] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, “Class-balanced loss based on effective number of samples,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9268–9277. [Online]. Available: https://openaccess.thecvf.com/content_CVPR_2019/html/Cui_Class-Balanced_Loss_Based_on_Effective_Number_of_Samples_CVPR_2019_paper.html
- [8] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [9] Datawrapper, “Create charts, maps, and tables,” Web-based data visualization platform, 2026. [Online]. Available: <https://www.datawrapper.de/>
- [10] M. Lin, Q. Chen, and S. Yan, “Network in network,” *arXiv preprint arXiv:1312.4400*, 2013.
- [11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: <https://www.jmlr.org/papers/v15/srivastava14a.html>
- [12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [13] TensorFlow, “Post-training quantization,” TensorFlow Model Optimization documentation, 2022. [Online]. Available: https://www.tensorflow.org/model_optimization/guide/quantization/post_training
- [14] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, “Tensorflow lite micro: Embedded machine learning for tinyml systems,” in *Proceedings of Machine Learning and Systems*, vol. 3, 2021. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2021/hash/6c44dc73014d66ba49b28d483a8f8b0d-Abstract.html
- [15] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf